INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE ENE - AGO 2026

CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

Patrones De Diseño

TÍTULO:
Prototipo

UNIDAD A EVALUAR:
UNIDAD 2

NOMBRE DEL MAESTRO:
MARIBEL GUERRERO LUIS

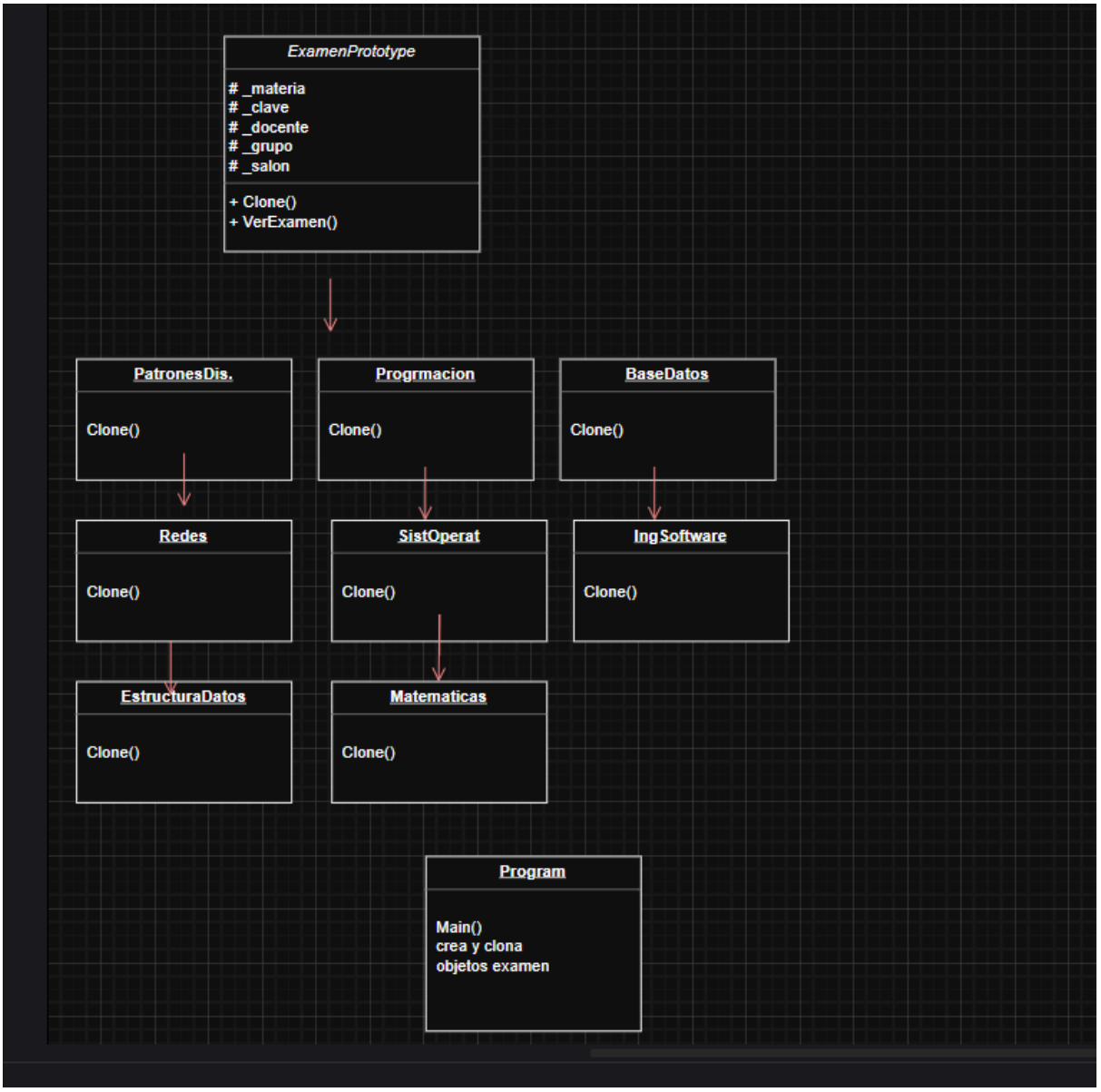
FECHA DE ENTREGA:
Tijuana, B.C. 010/03/2026

Introducción

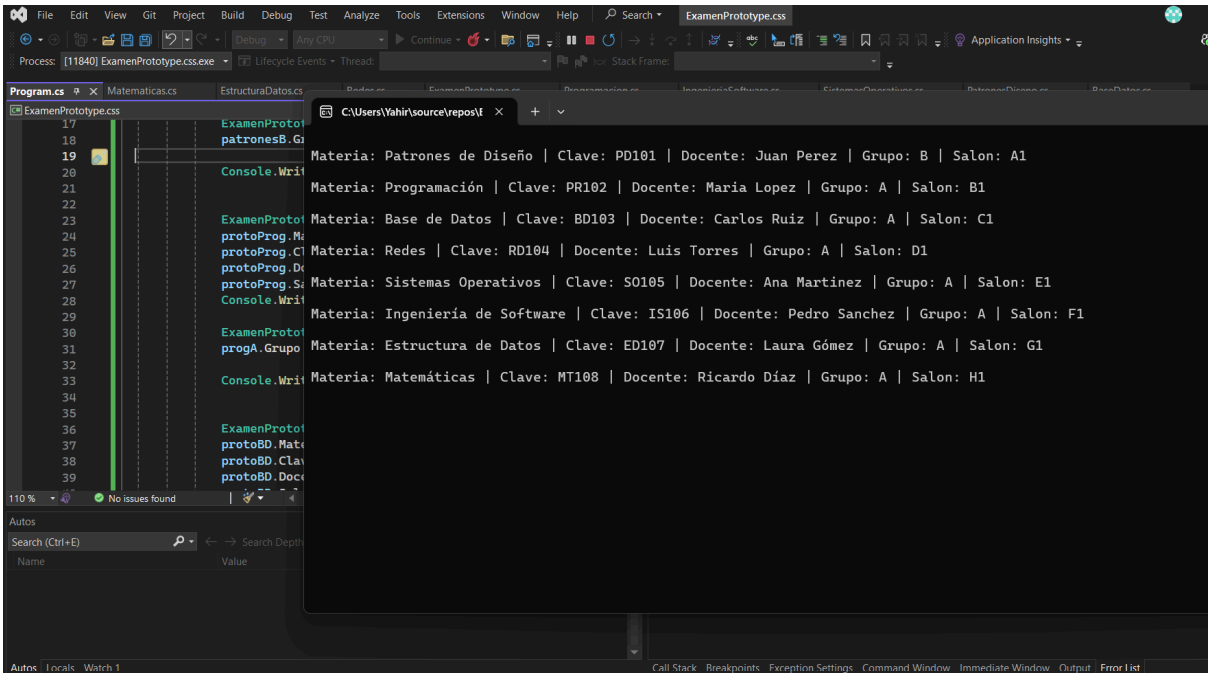
En esta práctica se desarrolló un programa utilizando el lenguaje C#, en el cual se implementa el patrón Prototype para la creación de exámenes de distintas materias dentro de una institución educativa. En el contexto del Instituto Tecnológico, un docente puede tener varios grupos de una misma materia, por lo que los exámenes pueden compartir ciertos atributos como el nombre de la materia, la clave de la asignatura, el docente y el salón, cambiando únicamente algunos datos como el grupo.

Para resolver este problema se utilizó herencia, clonación de objetos y atributos protegidos, permitiendo generar múltiples exámenes a partir de un prototipo base. De esta manera se evita repetir código y se facilita la creación de nuevos objetos con características similares.

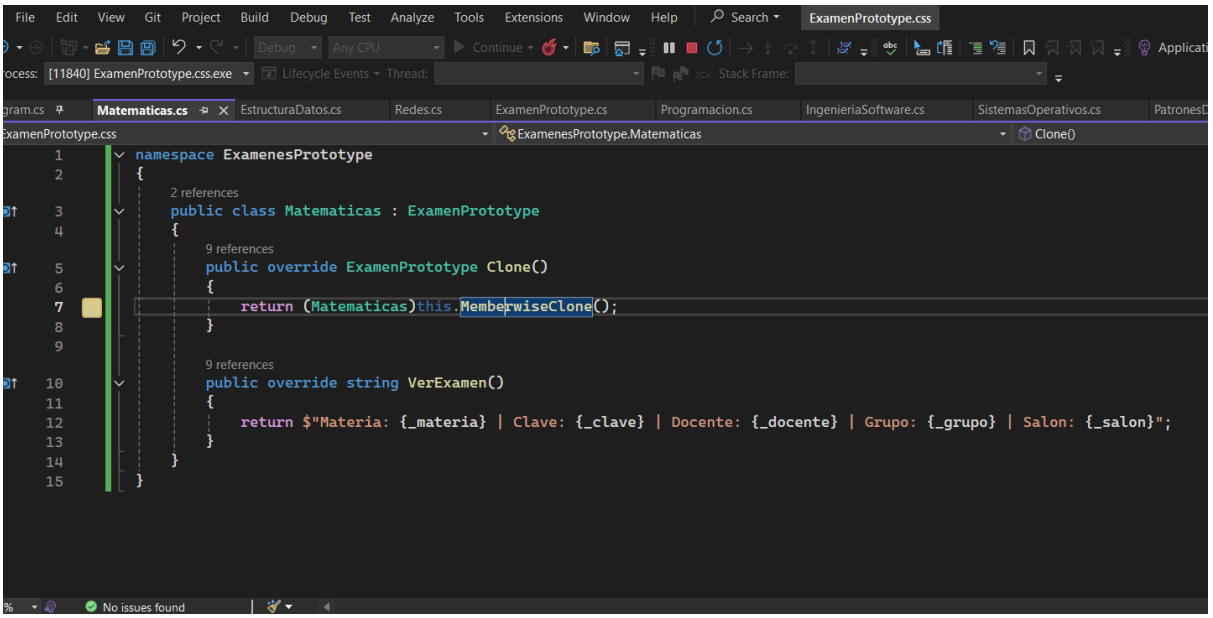
Diagrama UML



Código Ejecutable



En la imagen se muestra la ejecución del programa desarrollado en Visual Studio utilizando el patrón de diseño Prototype. En la consola se pueden observar los diferentes exámenes generados para varias materias como Patrones de Diseño, Programación, Base de Datos, Redes, Sistemas Operativos, Ingeniería de Software, Estructura de Datos y Matemáticas. Cada examen comparte atributos como la materia, clave, docente y salón, demostrando cómo mediante la clonación de un objeto prototipo se pueden crear múltiples instancias de exámenes para diferentes grupos sin necesidad de crearlos desde cero.



```
namespace ExamenesPrototype
{
    2 references
    public class EstructuraDatos : ExamenPrototype
    {
        9 references
        public override ExamenPrototype Clone()
        {
            return (EstructuraDatos)this.MemberwiseClone();
        }

        9 references
        public override string VerExamen()
        {
            return $"Materia: {_materia} | Clave: {_clave} | Docente: {_docente} | Grupo: {_grupo} | Salon: {_salon}";
        }
    }
}
```

```
namespace ExamenesPrototype
{
    2 references
    public class Redes : ExamenPrototype
    {
        9 references
        public override ExamenPrototype Clone()
        {
            return (Redes)this.MemberwiseClone();
        }

        9 references
        public override string VerExamen()
        {
            return $"Materia: {_materia} | Clave: {_clave} | Docente: {_docente} | Grupo: {_grupo} | Salon: {_salon}";
        }
    }
}
```

```
using System;

namespace ExamenesPrototype
{
    33 references
    public abstract class ExamenPrototype
    {
        protected string _materia;
        protected string _clave;
        protected string _docente;
        protected string _grupo;
        protected string _salon;

        8 references
        public string Materia { set => _materia = value; }
        8 references
        public string Clave { set => _clave = value; }
        8 references
        public string Docente { set => _docente = value; }
        8 references
        public string Grupo { set => _grupo = value; }
        8 references
        public string Salon { set => _salon = value; }
```

```
namespace ExamenesPrototype
{
    2 references
    public class Programacion : ExamenPrototype
    {
        9 references
        public override ExamenPrototype Clone()
        {
            return (Programacion)this.MemberwiseClone();
        }

        9 references
        public override string VerExamen()
        {
            return $"Materia: {_materia} | Clave: {_clave} | Docente: {_docente} | Grupo: {_grupo} | Salon: {_salon}";
        }
    }
}
```

```
namespace ExamenesPrototype
{
    2 references
    public class IngenieriaSoftware : ExamenPrototype
    {
        9 references
        public override ExamenPrototype Clone()
        {
            return (IngenieriaSoftware)this.MemberwiseClone();
        }

        9 references
        public override string VerExamen()
        {
            return $"Materia: {_materia} | Clave: {_clave} | Docente: {_docente} | Grupo: {_grupo} | Salon: {_salon}";
        }
    }
}
```

```
namespace ExamenesPrototype
{
    2 references
    public class SistemasOperativos : ExamenPrototype
    {
        9 references
        public override ExamenPrototype Clone()
        {
            return (SistemasOperativos)this.MemberwiseClone();
        }

        9 references
        public override string VerExamen()
        {
            return $"Materia: {_materia} | Clave: {_clave} | Docente: {_docente} | Grupo: {_grupo} | Salon: {_salon}";
        }
    }
}
```

```
namespace ExamenesPrototype
{
    2 references
    public class PatronesDiseno : ExamenPrototype
    {
        9 references
        public override ExamenPrototype Clone()
        {
            return (PatronesDiseno)this.MemberwiseClone();
        }

        9 references
        public override string VerExamen()
        {
            return $"Materia: {_materia} | Clave: {_clave} | Docente: {_docente} | Grupo: {_grupo} | Salon: {_salon}";
        }
    }
}
```

```
namespace ExamenesPrototype
{
    2 references
    public class BaseDatos : ExamenPrototype
    {
        9 references
        public override ExamenPrototype Clone()
        {
            return (BaseDatos)this.MemberwiseClone();
        }

        9 references
        public override string VerExamen()
        {
            return $"Materia: {_materia} | Clave: {_clave} | Docente: {_docente} | Grupo: {_grupo} | Salon: {_salon}";
        }
    }
}
```

Conclusión

En esta práctica aprendí a utilizar el patrón de diseño Prototype para crear objetos a partir de la clonación de un objeto base. Este patrón es muy útil cuando se necesitan crear muchos objetos que comparten características similares, ya que permite ahorrar tiempo y evitar repetir código.

Al desarrollar este programa entendí cómo aplicar conceptos importantes de la programación orientada a objetos como herencia, encapsulamiento y reutilización de código. También comprendí cómo se pueden crear diferentes tipos de exámenes para distintas materias utilizando un mismo modelo base y modificando sólo algunos atributos.